

Vehicle Wait-Time Monitoring System

Author: Yuvraj Singh

Role: Computer Vision Engineer Assignment

1. Problem Understanding

The objective of this assignment is to build a system that:

- Reads a video file
- Detects and tracks vehicles
- Monitors a predefined Region of Interest (ROI)
- Calculates how long each vehicle remains inside the ROI
- Displays bounding boxes, ROI, and wait time (MM:SS)
- Writes the processed output to a video file

The key challenge is not just detection, but maintaining consistent vehicle identity over time and computing wait duration reliably.

2. My Approach

I designed the solution as a structured processing pipeline:

Video → Frame Extraction → Detection → Tracking → ROI Check → Timer Logic → Annotation → Output Video

Each stage has a clear responsibility, which keeps the system modular and easier to debug or extend.

3. Detection and Tracking

Detection

I used **YOLOv8 (Ultralytics)** with pretrained weights. It provides a strong balance between speed and accuracy and integrates easily with tracking. Only the **car class** (COCO class ID 2) is filtered to focus on relevant objects.

Tracking

To compute wait time, consistent identity across frames is required. I used **ByteTrack** (via YOLO's tracking interface), which assigns a unique ID to each vehicle and maintains identity across frames.

Tracking ensures that the same vehicle is not treated as a new object every frame.

4. ROI Monitoring

The ROI is defined as a rectangular region.

A vehicle is considered inside the ROI if the **center of its bounding box** lies within the ROI boundaries. This approach is computationally efficient and stable for this use case.

For production-grade systems, an IoU-based ROI intersection method could further improve precision.

5. Wait-Time Calculation

For each tracked vehicle:

- When it first enters the ROI, its entry frame index is stored.
- On every subsequent frame inside ROI, wait time is computed as:

$(\text{current_frame} - \text{entry_frame}) / \text{fps}$

The result is formatted into MM:SS format and displayed above the bounding box.

Additionally, the system tracks:

- Number of vehicles currently inside ROI
- Maximum wait time observed so far

These metrics provide useful real-time insights.

6. Robustness Handling

To prevent unstable behavior, I introduced a **tolerance window**:

If a vehicle briefly leaves the ROI due to detection noise or slight movement, it is not removed immediately. Instead, it must remain outside the ROI for a fixed number of consecutive frames before being cleared.

This prevents:

- Timer flickering
- Sudden resets
- Boundary instability

7. System Design

The implementation follows a modular object-oriented structure:

- **ROI**Monitor – Handles ROI containment logic and drawing.
- **WaitTimeTracker** – Manages entry frames, tolerance logic, and active vehicle count.
- **VehicleWaitTimeSystem** – Orchestrates video processing, detection, tracking, annotation, and output writing.

This separation improves readability, maintainability, and scalability.

8. Dockerization

The project is containerized using Docker to ensure:

- Reproducibility
- Dependency isolation
- Ease of deployment

The application can be built and executed using:

```
docker build -t vehicle_timer .  
docker run -v <host_path>:/app vehicle_timer
```

This ensures consistent execution across environments.

9. Assumptions

- Static camera setup
- Fixed ROI
- Stable video FPS

- Vehicles predominantly moving forward

10. Possible Improvements

For real-world deployment, the following enhancements could be considered:

- GPU acceleration for real-time processing
- IoU-based ROI intersection
- Database storage of wait times
- Alert system for threshold violations
- Dashboard integration for analytics

Conclusion

The implemented system provides a robust and modular solution for monitoring vehicle wait times inside a defined ROI. It balances detection performance, tracking stability, and clean engineering design while being fully dockerized for reproducibility.

The focus was on building a reliable, maintainable, and production-oriented pipeline rather than just a proof-of-concept script.